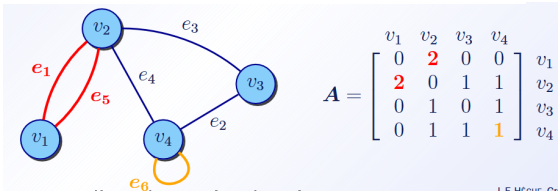
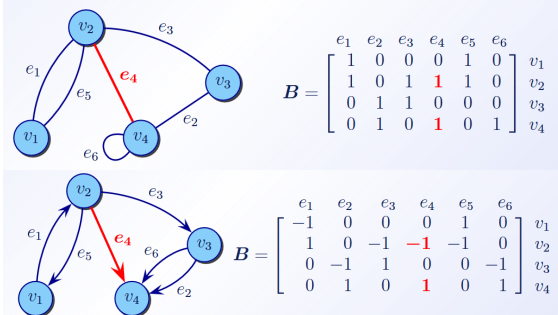


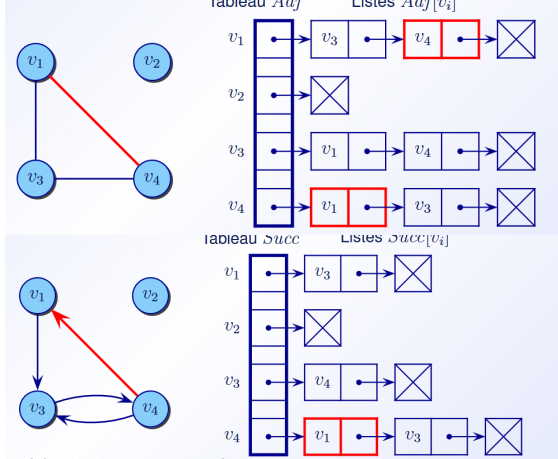
Matrice d'adjacence



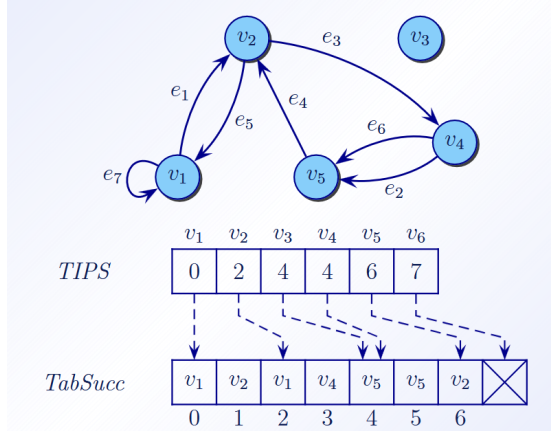
Matrice d'incidence (no loop)



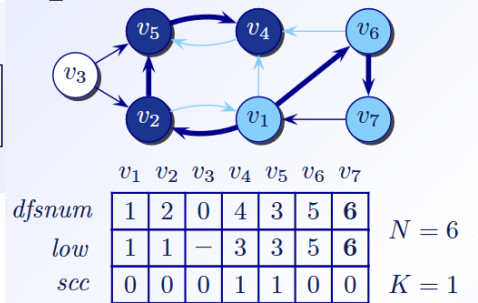
Tableaux de listes de successeurs



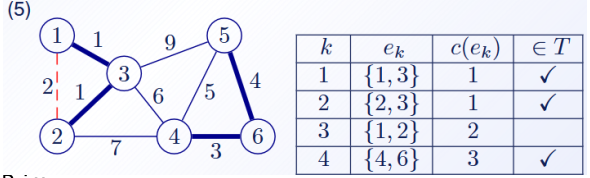
Tableaux compacts de successeurs



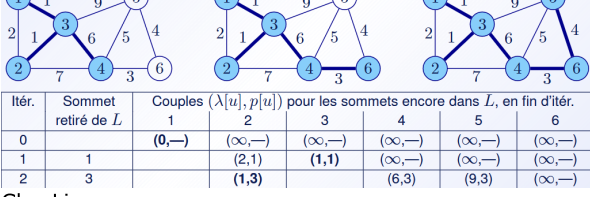
SCC_TARJAN



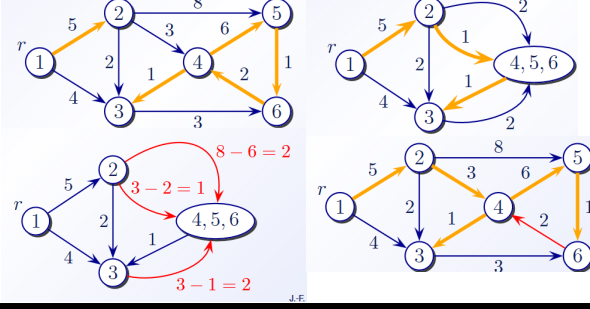
Kruskal



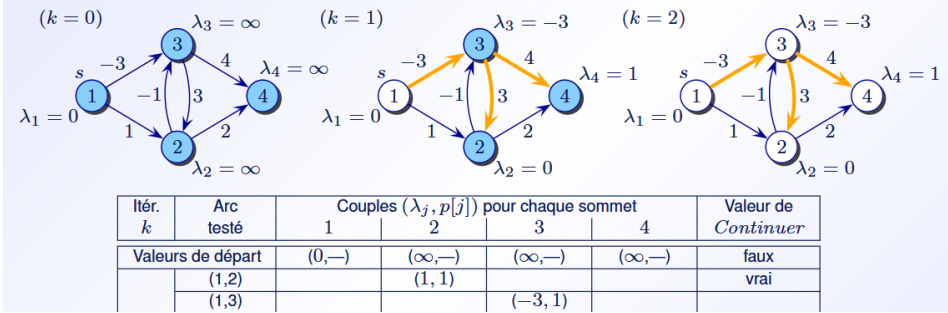
Prim



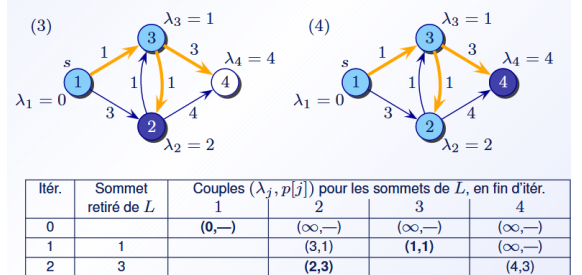
Chu-Liu



Bellman-Ford

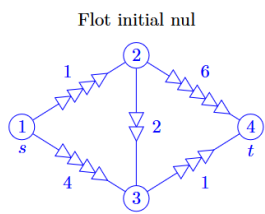


Dijkstra

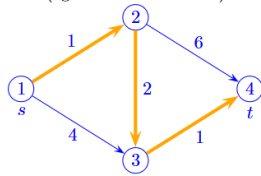


Every day GRE Exams

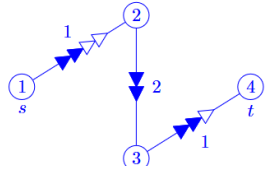




Premier réseau d'augmentation du flot
(égal au réseau initial)



Augmentation du flot (dans R^*)



Exploration de R^* (avec Dijkstra)
(arborescence des PCC en gras)

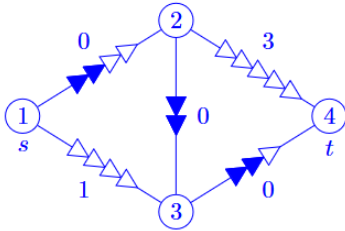
PCC (couples $(\lambda_i, p[i]) \forall i$)			
1	2	3	4
(0,-)	(1,1)	(3,2)	(4,3)

PCC de s à t : $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Capacité : 2

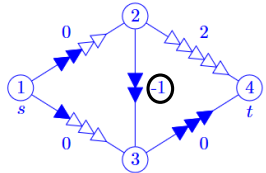
Coût unitaire (longueur du PCC) : 4

Flot au début de la 2^e itération



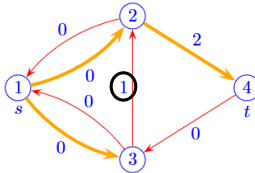
1. on commence avec un flot initial null
2. Construire un réseau d'augmentation R^*
3. Cherche un plus court chemin de s à t dans R^*
 - si trouver, répéter depuis (2)
 - (si coût) mettre à jour les cout en faisant $c_{ij} = c_{ij} + (\lambda_i - \lambda_j)$ (pour éviter les cout négative)
4. end

Flot au début de la 3^e itération

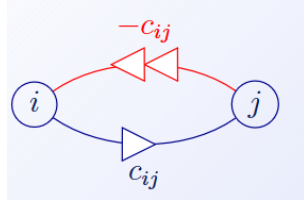


Augmentation du flot (dans R^*)

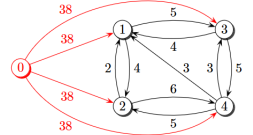
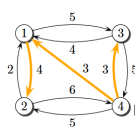
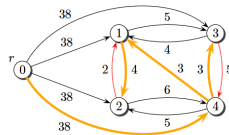
Réseau d'augmentation du flot



Exploration de R^* (avec Dijkstra)
(arborescence des PCC en gras)



Trick pour trouver une arborescence
recouvrante sans avoir de racin defini



Si plus que 1 arc $V_0 \rightarrow V_n$, alors forêt

Trouver si un graph a une arborescence recouvrant en temp linéaire

posé s = premier sommet traiter u = dernier sommet a finir son traitement

1. DFS
2. si $s = u \rightarrow$ Arborescence recouvrante a la racine s
3. sinon, parcour le graph depuis u , Si l'arborescence d'explo obtenue est recouvrante, problème résolu, si non sad

Transbordement : offre = - | demande = +

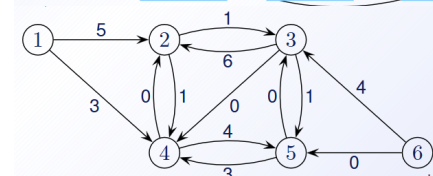
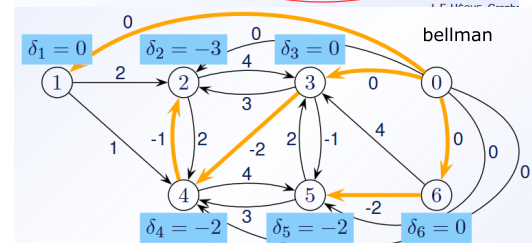
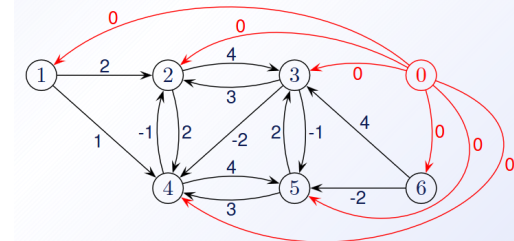
Type de problème → type de modélisation

- Problème de duo → couplage (logic)
- Problème de repartition de ressource → transbordement → flow max
- Problème de recherche de N chemin différent → flow max en utilisant la tech de split les sommets
- Problème de recherche de chemin en minisant le nombre de ressource global → recherche d'arbre recouvrant de poid min
- Trouver qu'elle arc est a bloquer / supprimer → recherche de Coupe → flow max
- Trouver les chemin pour maximiser les ressource passantes → flow max

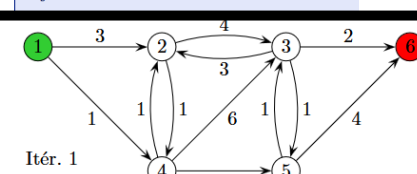
Vocabulaire

mots	def
Sommet	sommets
Arret	truc qui relie des sommets mais pas orienter
Arc	arret orienter
exploitation	just dire comment on pourrait s'en servir
Demi-degré	degré + ou -
Arbre	Graph sans cycle

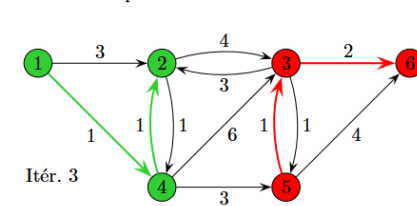
Johnson



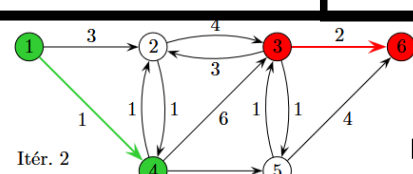
$$c'_{ij} = c_{ij} + \delta_i - \delta_j \quad \forall (i, j) \in E$$



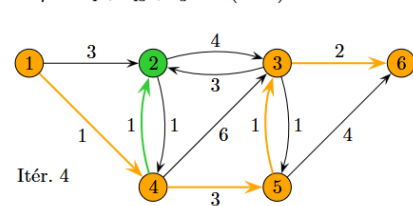
La longueur μ du meilleur chemin est initialisée à ∞ .
On traite la source $s = 1$ dans l'exploration en avant et la destination $t = 6$ dans l'exploration en arrière.



On traite le sommet 2 en avant : on met à jour μ lors du test de l'arc $(2, 3)$:
 $\mu = \lambda_2 + c_{23} + \delta_3 = 8 (< 9)$.
On traite le sommet 5 en arrière : on met à jour μ lors du test de l'arc $(4, 5)$:
 $\mu = \lambda_4 + c_{45} + \delta_5 = 7 (< 8)$.



On traite le sommet 4 en avant puis le sommet 3 en arrière.
Lors du test de l'arc $(4, 3)$ en arrière, 4 n'est plus dans Q_f : on met à jour μ
 $\mu = \lambda_4 + c_{43} + \delta_3 = 9 (< \infty)$.



On traite le sommet 5 en avant.
On s'arrête car 5 n'est plus dans Q_r .
Le PCC de $s = 1$ à $t = 6$ (en orange) a une longueur de $\mu = 7$ et passe par l'arc $(4, 5)$ (dernier arc ayant permis d'améliorer μ) : $1 \rightsquigarrow 4 \rightarrow 5 \rightsquigarrow 6$.

Théorème de Berge

en gros, il dit que si on a une chaîne donnée C , on peut lui ajouter un membre en faisant :

- crée un couplage d'arc non compris dans C
- fait $C \Delta M$



Recherche de couplage maximum

1. crée un couplage, peut importe le quel
2. Orient les arc :
droite $\rightarrow$ gauche
et gauche $\rightarrow$ droite
3. en partant du sommet "source" et allant vers un "pu" on optien une chaîne alternée C augmentente sur M
4. on fait $C \Delta M$

Other shit

- graph bipartite $K_{r,s}$ ($r + s$ sommet)
 - $nArc = r * s$ / couplage max cardinalité = $\min(r, s)$
- Si on a le couplage max d'un graph G , trouver le recouvrement $\rightarrow$ ajoute les sommet manquant pour formé l'arbre
- Si on a un arbre recouvrant min, trouver le couplage max = garder un arc pour tous sommet a plus de 1 connection

Dijkstra bidirectionnel

3 truc

(forward) fils de prio Q_f et mark λ

(backward) fils de prio Q_b (Q_r) et mark δ

μ = valeur de critère d'arrêt

pour chaque itération :

- itéré de 1 en avant, itéré de 1 en arrière
- **En avant** : si pour l'arc (i, j) , j a été **retiré** de la file Q_b alors mettre a jour si $\mu > \lambda_i + c_{ij} + \delta_j$ alors $\mu = \lambda_i + c_{ij} + \delta_j$
 - même chose en arrière si j a été retiré de Q_f
- Stopé si on trouve un sommet qui vas être traiter pour la deuxième fois (sois un sommet qui a déjà été retiré d'une des queue), la longueur du chemin final est de

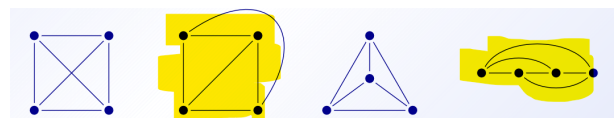
μ

Graph planaire

K_N = Graph complet a N sommets

Graph planaire topologique

En gros, c'est un graph planaire mais aucune de ses arretes ne se croises



ça découpe un plan en région

Formule d'euler

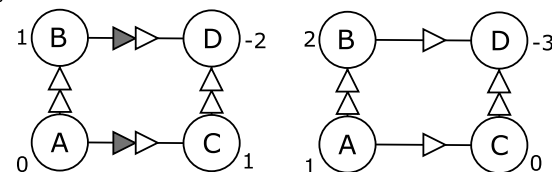
$$n - m + f = 2$$

$$n = n \text{ sommet} \mid m = n \text{ arrets} \mid f = n \text{ faces}$$

Effet :

- (1) tout graph planaire simple et connexe avec $n \geq 3$ et m arrets, on a $m \leq 3n - 6$
- (2) graph complet K_5 n'est pas planaire

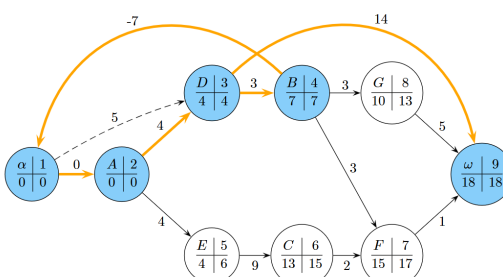
Dégager les flow min



Pour rajouter une contrainte du style : la tâche X doit être exécuter au plus tard N jours après Y
on ajoute un arc a poids partant de la tâche X vers Y de poids -N, cette boucle doit juste ne pas être un circuit abrochant
raison :

ça crée un chemin alternatif pour arriver a la tâche final qui prend en compte le nombre de jours maximal en gros

$$t_j = t_i + d_i \rightarrow t_j = t_i - N \text{ ce qui force la contrainte}$$



Tâche B doit commencer au plus 7 jours après le commencement du projet

Graph eulérien

- **Théorème** (Euler, 1736). Un graphe G non orienté et connexe est eulérien (autrement dit possède un cycle eulérien) si et seulement si tous ses sommets sont de degré pair.

Si G possède exactement 2 sommets de degré impair alors il admet une chaîne eulérienne (dont les extrémités sont les deux sommets de degré impair).
- Un graphe G orienté et connexe est eulérien (autrement dit possède un circuit eulérien) si et seulement si le demi-degré entrant de chaque sommet est égal à son demi-degré sortant.

Si l'égalité entre les demi-degrés entrant et sortant est vérifiée pour tous les sommets sauf deux pour lesquels l'écart entre les demi-degrés est égal à 1, le graphe admet un chemin eulérien (entre ces deux sommets de degré impair).

Le problème du postier chinois

en gros, on cherche un chemin qui passe par tous les sommet du graph mais avec une longueur min.

Version non orienter

dans un graph G

1. Posé W comme l'ensemble de tous les sommets de **degré impaire**
2. pour chaque sommet dans W trouver le chemin le plus court ver tous les autre sommet de W (longueur $\{i, j\}$)
3. Construire le graph complet H dont tous les sommets sont vienne de W , chaque arret a comme poid la longueur du chemin entre les sommets(longueur $\{i, j\}$)
4. Calculer un couplage maximum M de H avec le poid toltal minimal
5. pour chaque arret $\{i, j\}$ de M doubler, dans G , la plus court chain de i à j

Version orienter

Besoin que le graph sois fortement connext et sois sans circuit negatife

1. classer les sommets $\rightarrow b_i = A$ les sommet avec plus d'arc **entrant** que **sortant** | $b_j = B$ les sommet avec plus d'arc **sortant** que **entrant**
2. Pour chaque couple **(i,j)** dans $A \times B$ trouver le chemin le plus court de i à j (d_{ij})
3. Crée un graph Bipartie avec d'un côté les sommet de A et de l'autre les sommet de B mais, mettre le meme nombre de fois un sommet qu'il n'as de sommet en trop (sommement sortant ou entrant)
4. ajouter les arret entre i et j avec comme poid la distance entre i et j
5. Faire un couplage parfait de poid min
6. Dans G doublé toute les arc qui sont pris par le chemin le plus court entre i et j
7. crée le circuit eulerien

Transfromer un grph orienter non-eulerien

1. trouver les sommets avec un $|deg_+(v) - deg_-(v)| \geq 1$
2. les isolés
3. ajouter des arc pour équilibrée leur degrés
4. crée c'est arc sur le graph G

Tranfomer un grpah orienter en graph eulerier a moindre couts

Faut que le graph sois connext

1. en gros c'est un problème de transbordement
2. definir une offre/demande avec $(deg_-(i) - deg_+(i))/2$
3. mettre toute les arc a poids de 1 et a capacitée 1
4. Resoudre le problème de flow
5. les arc pris par la resolution sont a inversé

Preuve graph eulerien

denoté les 2 sommets expetion **s** et **t**

montre que le graph à un chemin eulerien mais pas un circuit

- Pour tous sommets sauf 2 (**s** et **t**)
 $\sum deg^- = \sum deg^+$
- Pour **s** et **t**, $|deg_+ - deg_-| = 1$

pour cercuit, on doit avoir

- Pour tous sommets
 $\sum deg^- = \sum deg^+$